

Thesis

Jerzy Rzeszutkowski

April 2026

1 Introduction

Question: To what extent does Neural Posterior Estimation improve the calibration and macro economical forecasting performance of a macroeconomic Agent-Based Model relative to traditional estimation methods?

The foundation for our analysis is the work of Poledna et. al. (2023), who developed an ABM model of the Austrian economy and demonstrated its forecasting performance. Their calibration method relies on a combination of cross sectional and time series data. Where the parameters for the government actions are estimated as coefficient of AR(1) processes. Moreover, my main parameter of interest, the fraction of income devoted to consumption (ψ), is estimated from input - output tables provided by Statistik Austria.

Their model was extended by the Wiese et. al. (2024) paper, where they incorporated Bayesian Method developed by Dyer et. al. (2024) to calibrate some parameters. They show that this method can improve the forecasting performance of the model, especially in the short term. Another application of the Dyer et. al. method is presented by Alves et. al. (2026), where they tune a labour market ABM. They choose three parameters and find that handcrafted summary statistics result in weaker posterior than NN learned ones.

2 Literature review

In the current state of the art macro economic modelling, there are two main approaches. The first one is based on Dynamic Stochastic General Equilibrium (DSGE) models, which are widely used in macroeconomics for policy analysis and forecasting. Popularized by Smets and Wouters (2007), these models are built on microeconomic foundations and incorporate rational expectations. However, they often rely on strong assumptions and may not capture the complexity of real-world economies.

The second approach, analyzed in this paper, is based on Agent-Based Models (ABMs). First used for economic modelling by Orcutt (1957), ABMs have gained popularity after the financial crisis of 2008. Presenting a more flexible framework than DSGE models, ABMs allow for the inclusion of heterogeneous agents, non-linear interactions, and emergent phenomena. Results by Farmer and Foley (2009), suggest that ABMs can provide valuable insights into

economic dynamics and policy implications, especially in situations where traditional models struggle to capture the complexity of the economy.

However, large ABMs often have a large number of parameters, which can make calibration and thus accurate forecasting, challenging. Traditional methods for calibrating ABMs include manual tuning, grid search, and optimization algorithms. These methods can be time-consuming and may not always yield accurate results. Recently, following the work of Grazini (2017), there has been growing interest in using Bayesian methods for calibration, which allow to estimate the (conditional) distribution of a parameter(s) of interest, instead of point estimate. They put forward three Approximate Bayesian Computation (ABC) methods for posterior estimation of parameters of ABM models. Bayesian modeling requires the knowledge of data likelihood function ($p(x|\theta)$), which is difficult to derive for large ABMs (Dyer et. al. 2024). The proposed ABC approaches to solve this problem by: (1) estimation of the likelihood function (parametric and non - parametric using e.g. kernel density approximations, which need MCMC, or similar sampler to find the posterior), (2) Likelihood free posterior estimation (Classical ABC - Grazzini et. al. 2017). All methods rely on simulation from the underlying model ($p(x|\theta)$), thus are a part of broader class of Simulation Based Inference methods (SBI). They show that posterior estimation can be beneficial for ABM calibration. It allows to capture the uncertainty in the parameters which is lost by point estimates given by previous methods, such as: Simulated Minimum Distance (SMD). SMD and similar methods produce as estimate $\hat{\theta}$ by minimalization of some loss function: $\hat{\theta} = \arg \min_{\theta \in \Theta} f(y, \theta)$, where y references the observed data. Mentioned ABC methods were extended by Platt (2021), with neural network approximation to the model's likelihood function, Lux (2018, 2021) with Monte Carlo methods relying on particle filters and Papamakarios et. al. (2019) with sequential neural likelihood (SNL) estimation technique. However, those ideas result in large required simulation budget (10^5 for low dimensional and 10^{7+} for higher dimensional) due to MCMC sequential nature and require stationarity assumption for simulator output (Dyer et. al. (2024)). Addressing those issues Dryer et. al (2024) propose two discriminative SBI methods: neural posterior estimation (NPE) (Papamakarios and Murray (2016), Lueckmann et al. [2017], Greenberg et al. (2019) and neural ratio estimation (NRE)(Hermans et al. 2020) for ABM calibration. NPE (method analysed in the rest of this paper) approximates the target posterior by neural network architecture: $q_{\phi}(\theta|x)$, where ϕ represent the trainable parameters. First used for conditional density estimation by Papamakarios and Murray (2016) with Mixture Density Network (MDN) (Bishop 1994) with fully parametrized gaussians as base components. The parameters ϕ are trained by backpropagation. This approach directly approximates the target (conditional) density.

Another option is to use Normalising Flows (NF) (Tabak and Vanden-Eijnden, 2010; Tabak and Turner, 2013; Rezende and Mohamed, 2015). It is based on transforming some base (simple) distribution (p_U , usually a standard normal) into the target one (p_X) (in (1), by a sequence of parametrized, invertible functions. It is based on the change of variable formula ($X = g(U) : g^{-1} = f$):

$$p_X(x) = p_U(f(x)) |\det J_f(x)| \quad (1)$$

Which can be unrolled to a composition (or flow) $f = f_n \circ \dots \circ f_2 \circ f_1$ where each f_i is differentiable and invertible with $g_i = f_i^{-1}$. Thus by sampling from U we can easily obtain $p_X(x)$. The functions f_i are parametrized by ϕ , and optimized by maximizing the data $(x_1, x_2, \dots, x_R \sim p_X)$ likelihood under the model:

$$\hat{\phi} = \arg \max_{\phi} \sum_{i=1}^R \log q_{\phi}(x_i) \quad (2)$$

NF modeling have two independent choices: the dependency structure and scalar transform. The first one specified how marginal variables of x (given it is a vector) influence each other. Scalar transform dictates the way values are propagate through the network (see (3) below). It can be extended to conditional density estimation. The main approach used for NPE is based on Masked Autoregressive Flow (MAF)(Papamakarios et. al. 2017). MAF mainly specifies the autoregressive nature of the flow and is constructed using gaussian MADE (Germain et. al. 2015) blocks as the flow functions (f_i). MADE is a NN that approximates a joint density: $q_{\phi}(X_1, \dots, X_n)$ by a series of conditionals: $q_{\phi}(\cdot) = \prod_i^n p(x_i | x_{j < i})$ (autoregressive). It uses binary masks to compute all conditionals in a single forward pass instead of coupled neural networks. For example, for a dense layer $h = Wx + b$, a MADE would introduce a mask matrix W , that restrict impossible connections e.g.: $p(x_2 | x_1)$ only depends on $p(x_1)$. Thus we have a MADE dense layer: $h = (M \odot W)x + b$. With $\odot$ being element wise multiplication. MADE networks can be used for density estimation, then to compute $p(x_1, \dots, x_n)$, we draw $\{u_1, \dots, u_n\}$ from some simple predefined distribution and run the network forward. However, as MAF experiments suggests, stacking these blocks can improve density estimation. In that case we can use the resulting joint distribution from one MADE block as sampling distribution for u 's for the next ones.

MAF models the conditional densities as Gaussian: $p(x_i | x_{<i}) \sim \mathcal{N}(\mu_i, \exp(\alpha_i))$, where μ_i and α_i depend on $x_{<i}$. Thus, given draws for $u_i \sim \mathcal{N}(0, 1)$ we can simulate (propagate) x_i by:

$$x_i = \mu_i + u_i \cdot \exp \alpha_i \quad (3)$$

The equation (3) is used to propagate both ways: sampling (draw u , obtain x) and density evaluation: given x compute corresponding u (invertibility). It is an example of an affine transform, and can be improved upon. Durkan et. al. () suggest to replace this step by rational - quadratic spline transform. Or more generally splines. Thus the name Neural Spline Flow (NSF)

Regardless of the architecture chosen, the learning for sample $\{\theta_i, x_i\}_{i=1}^N$ proceeds with minimizing the cross entropy (equivalent to minimizing KL divergence between true posterior: $p(\theta|x)$ and learned posterior: $q_{\phi}(\theta|x)$):

$$\mathcal{L}(\phi) = -\frac{1}{N} \sum_i \log q_{\phi}(\theta_i | x_i) \quad (4)$$

One usually uses standard deep learning techniques such stochastic gradient descent with optimizers such as ADAM (e.g. Dyer et. al. 2024) to update the parameters ϕ .

If the entire sample $\{\theta_i, x_i\}$ is generated by draws from the prior and used to train the NPE, we have a single round inference. This approach gives amortized posterior. That means that we can infer a conditional posterior for arbitrary observation x_{obs} . However, in many cases one is interested in posterior for one particular realization e.g. GDP trajectory, which can be restricted to a smaller parameter space. Thus, it may be beneficial to automatically update the prior based on the previous estimated posterior and generate pairs according to the new distribution. This "sequential" approach to SBI methods has been given some attention, beginning with the work by Papamakarios and Murray (2016). They introduce the first sequential NPE (SNPE), which I will refer to as SNPE_A. They set the initial prior ($p(\theta)$) to a simple gaussian and begin standard NPE training of $q_\phi(\theta|x)$ (modelled via gaussian MDN) by loss derived from (2) with samples from prior. After convergence, we set proposed prior ($\tilde{p}(\theta)$) to learned density, and train the next round by sampling from the proposed prior. This repeats for N rounds, after which we return the learned density: $q_\phi(\theta|x)$. This method allows for more efficient use of simulation budget. To account for change of sampling distribution, while learning the correct conditional density: $p(\theta|x) \propto p(\theta)p(x|\theta)$, the authors employ importance weighting when updating the proposed prior between rounds:

$$\hat{p}(\theta|x) \propto \frac{p(\theta)}{\tilde{p}(\theta)} q_\phi(\theta|x) \quad (5)$$

Where, the fraction represent the importance weighting factor and $q_\phi(\theta|x)$ is trained on sample $\{\theta_i, x_i\}$ from proposed prior. This transformation is possible due to analytical forms for both densities.

Lueckmann et. al. (2017) proposes an improvement to SNPE. Instead of learning a different posterior and transforming after, they incorporate importance weighting directly into the loss function of (4) and add a calibration kernel K_τ :

$$\mathcal{L}(\phi) = -\frac{1}{N} \sum_i \frac{p(\theta)}{\tilde{p}(\theta)} K_\tau(x_i, x_{\text{obs}}) \log q_\phi(\theta_i|x_i) \quad (6)$$

The calibration kernel is used to measure the distance between the observed x and simulated points x_i . In other words, the posterior should focus on x_i that are in some sense close to x_{obs} . Their methods allows for non gaussian priors and proposals, to which the previous method was restricted. However, may yield higher variance in estimation (Papamakarios et. al. 2019, Lueckemann et. al.2017).

The most robust approach was introduces by Greenberg et. al. (2019). Their algorithm: SNPE_C incorporates automatic posterior transformation (APT) for SNPE. APT solves both problems outlines above. It allows for a great variety of priors and proposals, while keeping the variance stable. It achieves this result by defining and training a transformed posterior ($\tilde{q}_\phi(\theta|x)$):

$$\tilde{q}_\phi(\theta|x) = q_\phi(\theta|x) \frac{\tilde{p}(\theta)}{p(\theta)} \frac{1}{Z} \quad (7)$$

Where Z is a normalization constant. APT uses $\tilde{q}_\phi$ to represent both true posterior and proposals at each round. Thus after a round of training (with standard nn loss: (4) - with $\tilde{q}_\phi$

in the logarithm) we can recover new proposed prior by: $\tilde{p}(\theta) = q_\phi(\theta|x_{\text{obs}})$, without analytical transformation. However, Z is an integrated expression, which may not be traceable for general densities. Authors introduce atomic proposal and show that then one can frame the APT inference procedure as finite classification problem, by asking which parameter values generated this observation. What turns continuous integral into a finite sum. Based on experiment in the paper, authors find that SNPE_C is more flexible than SNPE_A, more stable than SNPE_B. Moreover, they compare it with SNL method, and find similar or better performance, even on problems which would favor SNL.

NPE and similar methods are based on simulation outputs for some complex, stochastic models. Especially for ABM, resulting vector is a high dimensional time series, thus it can be beneficial to lower the input dimension for the NPE network (e.g. Dyer et. al. 2024) or incorporate some prior knowledge about output (e.g. exchangeability, dependence) as in time series. This can be done in two main approaches: selecting a set of summary statistics that describe the output, using a NN to learn informative features from data directly. The first is especially relevant for classical ABC approaches, where the algorithm mainly depends on the choice of distance function and summary statistics (e.g. Greenberg et. al. 2019). Thus some literature exists for choosing and testing summary statistics (e.g. Prangle 2018). However, this method can lead to, hard to quantify, losses of information (e.g. Radev et. al. 2020).

The other approach relies on constructing an auxiliary NN that maps simulation output x to a fixed size vector of learned statistics: $s_l(x)$. It is trained together with the density network from the same loss propagation. Many different choices exist for this network, from simple MLP for finance models (Stilmann et. al. 2023), 1D CNN (Radev et. al. 2020), standard CNN used for images (Greenber et. al. 2019) or LSTM recurrent neural networks (Gers et. al. 2000, Dyer et. al. 2024). These architectures can deal with time series data. Another example is by Radev et. al. (2020) with BayesFlow architecture, which is based on invertible neural networks needed for iid. simulation output.

After computing the approximate joint posterior distribution using SNPE or a similar methods, one would like to verify whether the result can be trustworthy. The first easy step, is to condition on the known data (x_{obs}) and check whether the posterior includes them (assigns non zero propability). One can extend this method to check whether parameters from the posterior give rise to sensible simulation result - close to observed data. (named: Posterior Predictive Checks). While both are useful, they do not provide any mathematical notion of fit. One of the first formal tests was introduced by Geweke (2004). Author suggests to check whether the posterior sampler (aka. approximation) reconstructs the joint distribution by constructing a Gibbs sampler from the estimated posterior: $q(\theta|x)$ and model's likelihood function: $p(x|\theta)$. If the inference is correct, then the sampler will produce accurate joint samples indistinguishable from those generated directly from the prior: $p(\theta, x) = p(x|\theta)p(\theta)$ (we can sample $p(x, \theta)$ both components are known). Z scores can be used to perform a formal test. However, as Talts et. al. (2018) note, the sampler can take a while to converge and validity of z - scores rely on this property.

Cook et. al. (2006) introduces another crucial idea, instead of comparing summaries of the joint distribution using z - scores, we check whether the true value behaves as a draw from its own posterior. This novel approach uses rank statistics (estimated quantiles) to perform a formal check dimension wise. If model and posterior is correct, theses statistics should have a uniform distribution. Authors suggest to formally check this by transforming resulting quantiles into z - scores by the inverse normal CDF function. Moreover, the shape of the resulting histogram can give a hint about the source of miss specification. While correct on some experiments, Talts et. al. shows that this procedure can inhibit bias when using MCMC sampling. To address this issue, they introduce Simulation Based Calibration (SBC), which is also based on rank statistics, but do now assume independent draws from the posterior sampler. Instead of proving the result for quantiles (continuous uniform), they show that the rank statistics for one dimensional variable follow a discrete uniform distribution.

3 Methodology

We will try to answer the question, whether Neural Posterior Estimation improves the calibration and forecasting performance of ABM models. As the underlying simulation engine, we use ABM model developed by Poledna et al. (2023), on Italian economy in ($T_1 = 2010Q1$) and calibrated using data from 1995Q1 to 2009Q4. The calibration method is described in the original paper.

Since our analysis is based on posterior estimation and forecasting performance, there are three main steps:

1. Posterior estimation using NPE method on hypothetical trajectories from 2010Q1 to 2016Q4.
2. Validation of the posterior using approximate Bayesian technique: Simulation Based Calibration.
3. Evaluation of forecasting performance of the model with parameters obtained from NPE.

3.1 Sampling

3.2 Posterior Estimation

Symbol	Name	Prior range
ψ	fraction of income devoted to consumption	
α^G	Autoregressive coefficient for government consumption	
α^E	Autoregressive coefficient for exports	
β^E	Scalar constant for exports	
ξ^γ	Weight of economic growth	

Table 1: Parameters to estimate, convention the same as in Poledna et. al paper

The simulation includes a variate of parameters, some of them stay constant, some vary over time. In order to answer the research question, we focused on five constant ones, that we think have significant influence over GDP growth. Moreover we, tried to identify those, for which standard calibration (used by Poledna et. al.) might be difficult. Due to lack of sufficient data or the nature of the parameter itself. Our choice of parameters is presented in Table. 1 and explanation together with GDP decomposition is below:

$$Y = \text{Consumption} + \text{Investment} + \text{Gov. expenditure} + \text{Net Exports} \quad (8)$$

ψ - It reflects the fraction of households income devoted to consumption. We decided to include it in our analysis, since consumption is one of the major drivers of GDP growth. While at the same time, the fraction can be quite challenging to estimate.

α^G - Second parameter dictates the persistence of GDP. Since Government expenditure is a significant portion of Austria GDP (more than 50% in 2024), this parameter can identify this relation.

We want to find:

$$p(\theta|x_{\text{obs}}) = \frac{p(x_{\text{obs}}|\theta)p(\theta)}{p(x_{\text{obs}})} \quad (9)$$

With x_{obs} referring to the observed trajectory of GDP growth from 2010Q1 to 2016Q4. However, the left hand side of the above expression is intractable and difficult to derive (Dyer et. al. 2022, 2024). Thus, we use SNPE_C method to approximate this posterior by running many simulations with parameters from the proposed priors.

The inference procedure, as described by Dyer et. al. starts by choosing appropriate priors for the choose parameters. Following the modeling choice of Alves et. al., and lack of strong signals to deviate, we decided to use uniform priors ($p(\theta)$) with bounds presented in table 1.

With the given priors, we begin the sequential NPE estimation:

1. Draw N samples from the current proposed prior (original prior in the first round): $\{\theta_i\}_i^N$
2. For each draw, obtain x_i from the simulation, with the chosen parameters replaced by the draw θ_i , and the rest obtained by original calibration. Due to stochasticity of the simulator, we decide to average M runs for each parameter draw (eq: (10))
3. Perform single round NPE estimation according to loss given by SNPE_C based on (see dimensionality reduction)
 - (a) A set of handcrafted statistics, yielding: $\{\theta_i, s(x_i)\}_i^N$
 - (b) Feature selection made by an auxiliary (embedding network: H): $\{\theta_i, H(x_i)\}_i^N$
4. (When using handcrafted statistics) Remove extreme outliers by the IQR approach: below $Q1 - m \cdot IQR$ or above $Q3 + m \cdot IQR$. We used $m = 8$.
5. Update the proposed prior by: $\tilde{p}(\theta) = q_\phi(\theta|x_{\text{obs}})$

Summary statistics	S1	S2	S3	S4
Series mean	-	-	-	-
Series standard deviation	-	-	-	-
AR(1) coefficient	-	-	-	-
AR(2) coefficient	-	-	-	-
Lag k correlation	-	-	-	-
Skewness	-	-	-	-
Kurtosis	-	-	-	-
q - percentile	-	-	-	-
Recession count	-	-	-	-

Table 2: Summary statistics

$$x_i = \frac{1}{M} \sum_{m=1}^M x_i^m \quad (10)$$

3.2.1 Dimensionality reduction

Next step is to choose a dimensionality reduction method. Since our NPE model learns (and then inverts) the map $x \in \mathbb{R}^{T_{\text{train}}}$; $T_{\text{train}} = T_2 - T_1$, it is useful to lower the input dimension to prevent erratic optimization and overfitting. We compare two main approaches:

- a. Use a hand crafted set of statistics: $s(x_i) : \mathbb{R}^{T_{\text{train}}} \rightarrow \mathbb{R}^d$. With d being the number of statistics. We decided to check a few combinations of summary statistics, all of them are computed over log difference: $\ln X_{t+1} - \ln X_t$ to account for possible non - stationarity.
 - i. Ten summaries chosen by Dyer et. al. (2024) and ALves et al. (2026): mean, variance, max, min, q_p , γ_k . For $p \in \{.25, .5, .75\}$ and $k \in \{1, 2, 3\}$ ($d_1 = 10$).
 - ii. The nature of our output (GPD) suggest to incorporate some measure for yearly cycles and shape of the distribution. Thus we add yearly correlation, skewness and recession count; and remove autocorrelation for $k \geq 2$ and quartiles except for the median ($d_2 = 9$).
- b. Another approach is to let the network learn the features directly, during the training process. Here we also compare a few choices:
 - i. CNN network with 1D convolutional layers on raw log - differenced output.
 - ii. CNN network with 2 channels: log level, percentage growth. Combined with MLP for hand crafted summary statistics.
 - iii. RNN network used by Dyer et. al. and the labour market analysis.

CNN networks are inspired by the research by Radev et al. (2020).

With both methods, we transform the posterior estimation problem from $p(\theta|x)$ to $p(\theta|s(x))$ where $s(x)$ references both the crafted and learned summaries. The posterior is estimated using extension of normalizing flows (Tabak and Vanden-Eijnden, 2010; Tabak and Turner, 2013; Rezende and Mohamed, 2015) called Masked Autoregressive Flow (Papamakarios and Murray, 2016; Lueckmann et al., 2017; Papamakarios et al., 2019; Greenberg et al., 2019).

3.3 Validation of the posteriors

As with other Bayesian Methods, evaluating the validity of the posterior is a crucial step in the inference process. Since we construct a conditional posterior used for forecasting, we use historical data: from 1995Q1 to 2009Q4, to obtain required summary statistics: $s(x_{obs}) = s(x_{1995Q1:2009Q4})$. We use standard rejection sampling to obtain draws from the conditional posterior. For each option, we run the following diagnostics, if option fails, it is not considered further:

1. Pairplot check - assess whether the conditional posterior includes the values given by standard calibration (by Poledna et. al.)
2. Posterior Predictive Check - run the simulation N' times with parameter samples from the conditional posterior, compute distribution for statistics (either hand - crafted or through a neural network) and check whether the observed ones fall onto the support.
3. Simulation Based Calibration - the formal diagnostics procedure as described by Talts et. al. (2018). The follow Algorithm 1 from that paper, with $L =$, and repeated $N =$ times:
 - i. Draw a prior sample: $\tilde{\theta} \sim p(\theta)$
 - ii. Draw a simulated data set: $\tilde{x} \sim p(x|\theta)$
 - iii. Draw posterior samples: $\{\theta_1, \dots, \theta_L\} \sim q_\phi(\theta|\tilde{x})$
 - iv. For each one - dimensional random variable do:
 - i. Compute the rank statistic as in (11)
 - ii. Update the histogram with the rank statistic

With the obtained histogram we check for deviations from uniformity.

$$r\left(\{f(\theta_i)\}_i^L, f(\tilde{\theta})\right) = \sum_{i=1}^L \mathbb{I}\left[f(\theta_i) < f(\tilde{\theta})\right] \in [0, L] \quad (11)$$

3.4 Forecast validation

For each method (option) that passed validation we compute the out of sample forecast from $T_{\text{begin}} = 2016Q1$ till $T_{\text{end}} = 2020Q4$ with the analyzed parameters taken to be the empirical

mean of $N = 1000$ samples from the conditional posterior. We compute the RMSE for each forecast length (indexed by quarters), $T \in [0, T_{\text{end}}]$:

$$\{RMSE_T\}_{T=0}^{T_{\text{end}}} = \sum_{t=T_{\text{begin}}+1}^T (x_t - y_t)^2 \quad (12)$$

4 Results

5 Summary

6 References

1. Poledna, S., Miess, M. G., Hommes, C., and Rabitsch, K. (2023). Economic forecasting with an agent-based model. *European Economic Review*, 151:104306.
2. Papamakarios, G., Pavlakou, T., & Murray, I. (2017). Masked autoregressive flow for density estimation. *Advances in neural information processing systems*, 30.
3. Dyer, J., Cannon, P., Farmer, J. D., and Schmon, S. (2024). Black-box Bayesian inference for agent-based models. *Journal of Economic Dynamics and Control*, 161:104556.
4. Alves, M. L., Dyer, J., Farmer, D., Wooldridge, M., & Calinescu, A. (2026). Neural Network-Based Parameter Estimation of a Labour Market Agent-Based Model. *arXiv preprint arXiv:2602.15572*.
5. Jan-Matthis Lueckmann, Pedro J Goncalves, Giacomo Bassetto, Kaan Ocal, Marcel Nonnenmacher, and Jakob H Macke. Flexible statistical inference for mechanistic models of neural dynamics. *Advances in Neural Information Processing Systems*, 30, 2017.
6. George Papamakarios, David Sterratt, and Iain Murray. Sequential neural likelihood: Fast likelihood-free inference with autoregressive flows. In *The 22nd International Conference on Artificial Intelligence and Statistics*, pages 837–848. PMLR, 2019.
7. David S. Greenberg, Marcel Nonnenmacher, and Jakob H. Macke. Automatic posterior transformation for likelihood-free inference. *36th International Conference on Machine Learning, ICML 2019, 2019-June:4288–4304*, 2019.
8. Esteban G Tabak and Eric Vanden-Eijnden. Density estimation by dual ascent of the log-likelihood. *Communications in Mathematical Sciences*, 8(1):217–233, 2010.
9. Esteban G Tabak and Cristina V Turner. A family of nonparametric density estimation algorithms. *Communications on Pure and Applied Mathematics*, 66(2):145–164, 2013.
10. Danilo Rezende and Shakir Mohamed. Variational inference with normalizing flows. In Francis Bach and David Blei, editors, *Proceedings of the 32nd International Conference*

- on Machine Learning, volume 37 of Proceedings of Machine Learning Research, pages 1530–1538, Lille, France, 07–09 Jul 2015. PMLR. URL <https://proceedings.mlr.press/v37/rezende15>.
11. Glielmo, A., Devetak, M., Meligrana, A., & Poledna, S. (2025). Beforeit. jl: High-performance agent-based macroeconomics made easy. arXiv preprint arXiv:2502.13267.
 12. Talts, S., Betancourt, M., Simpson, D., Vehtari, A., & Gelman, A. (2018). Validating Bayesian inference algorithms with simulation-based calibration. arXiv preprint arXiv:1804.06788.
 13. Wiese, S., Kaszowska-Mojša, J., Dyer, J., Moran, J., Pangallo, M., Lafond, F., ... & Farmer, J. D. (2024). Forecasting macroeconomic dynamics using a calibrated data-driven agent-based model. arXiv preprint arXiv:2409.18760.
 14. Grazzini, J., Richiardi, M. G., & Tsionas, M. (2017). Bayesian estimation of agent-based models. *Journal of Economic Dynamics and Control*, 77, 26-47
 15. Cook, S. R., Gelman, A., & Rubin, D. B. (2006). Validation of Software for Bayesian Models Using Posterior Quantiles. *Journal of Computational and Graphical Statistics*, 15(3), 675–692. <https://doi.org/10.1198/106186006X136976>
 16. Radev, S. T., Mertens, U. K., Voss, A., & Köthe, U. (2020). Towards end-to-end likelihood-free inference with convolutional neural networks. *British Journal of Mathematical and Statistical Psychology*, 73(1), 23-43.
 17. Lux, T. (2018). Estimation of agent-based models using sequential Monte Carlo methods. *Journal of Economic Dynamics and Control*, 91, 391-408.
 18. Germain, M., Gregor, K., Murray, I., & Larochelle, H. (2015, June). Made: Masked autoencoder for distribution estimation. In *International conference on machine learning* (pp. 881-889). PMLR.
 19. Grazzini, J., & Richiardi, M. G. (2013). Consistent estimation of agent-based models by simulated minimum distance. *LABORatorio R. Revelli working papers series*, 130.
 20. Papamakarios, G., Sterratt, D., & Murray, I. (2019, April). Sequential neural likelihood: Fast likelihood-free inference with autoregressive flows. In *The 22nd international conference on artificial intelligence and statistics* (pp. 837-848). PMLR.
 21. Prangle, D. (2018). Summary statistics. In *Handbook of approximate Bayesian computation* (pp. 125-152). Chapman and Hall/CRC.
 22. Stillman, N. R., Baggott, R., Lyon, J., Zhang, J., Zhu, D., Chen, T., & Vytelingum, P. (2023, November). Deep calibration of market simulations using neural density estimators and embedding networks. In *Proceedings of the Fourth ACM International Conference on AI in Finance* (pp. 46-54).

- 23. Radev, S. T., Mertens, U. K., Voss, A., Ardizzone, L., & Köthe, U. (2020). BayesFlow: Learning complex stochastic models with invertible neural networks. *IEEE transactions on neural networks and learning systems*, 33(4), 1452-1466.
- 24. Gers, F. A., Schmidhuber, J., & Cummins, F. (2000). Learning to forget: Continual prediction with LSTM. *Neural computation*, 12(10), 2451-2471.
- 25. Durkan, C., Bekasov, A., Murray, I., & Papamakarios, G. (2019). Neural spline flows. *Advances in neural information processing systems*, 32.
- 26. Cook, S. R., Gelman, A., & Rubin, D. B. (2006). Validation of software for Bayesian models using posterior quantiles. *Journal of Computational and Graphical Statistics*, 15(3), 675-692.
- 27. Geweke, J. (2004). Getting it right: Joint distribution tests of posterior simulators. *Journal of the American Statistical Association*, 99(467), 799-804.